

CPU ML Engineering

Five sample tasks

Long-horizon Harbor coding environments.

This document goes over how the folder is structured, what the tasks are, why they are relevant and how many we can make. More in-depth information is given with each sample task in its document.

Pack identification

Field	Overview
Category	CPU ML engineering
Package	Five self-contained Harbor task samples
Task format	Editable workspace, instructions, tests, verifier and reference solution per task
Source documents	Detailed task information is provided in each accompanying sample document

Description

This data product delivers **five long-horizon CPU ML-engineering tasks**: Agentic environments built on real public data and realistic production failure modes. In each, the agent inherits a broken ML pipeline and must do the **full engineer's job**, investigating the evidence, identifying the underlying defect, repairing the system, deploying through a gated release service, recovering from a staged failure, and driving the change to submission. Grading is deterministic and by-value against a hidden slice, so reward tracks genuine correctness rather than a plausible-looking patch and shortcut solutions do not score. A pass is real capability signal, not a guess that happened to look right.

The value to a lab is measured, spread difficulty rather than asserted difficulty. The five tasks are deliberately **two-tiered**: three are calibrated so a current frontier model solves them at a healthy but non-trivial rate, giving usable training and evaluation signal today, while two are ultra-hard, **deep-SWE-style tasks** the current frontier does not solve at all (0 of 4 attempts), positioned as headroom above the frontier where the unsolved rate is itself the value. Difficulty is verifiable, not claimed: every task ships a gold reference solution alongside its grader, so a lab can confirm each task is genuinely solvable, and a zero frontier pass rate reflects real difficulty rather than a broken or unfair task. All five share a hardened, isolated architecture (protected verifier state, privilege-dropped candidate execution, and leak-guarded packaging) that keeps difficulty honest and grading resistant to gaming.

Contents

Five task packages with separate rollout traces and **task submission documents**. The detailed task-level descriptions, why the model failed, prompts, environments and artifacts are documented with each sample. Grading is deterministic, behavioral, isolated, and control-checked, rewarding genuine correctness rather than plausible patches while resisting shortcut solutions. The tasks also have dense reward signals rather than a single pass/fail bit.

Technical domains / relevant benchmarks

Designed for frontier AI labs that need difficult, execution-grounded tasks for training and evaluating long-horizon coding and ML agents. The tasks complement **Terminal-Bench 2.1**, **DeepSWE v1.1**, **SWE-Bench Pro/Verified** and **MLE-Bench Revised** while adding production-ML failure diagnosis and stateful release operations.

Agents must explore unfamiliar repositories, diagnose failures from real data, repair the implementation and complete a gated deployment workflow under partial observability.

The tasks combine DeepSWE- and SWE-bench-style repository engineering, Terminal-Bench-style autonomous terminal execution and MLE-bench-style ML engineering. Their constrained tools, hidden state, ordered operations and end-state verification also exercise capabilities relevant to T-bench-style stateful tool use, while focusing on autonomous production-ML incidents rather than simulated user conversations.

Capacity

Next 3 weeks	Next 6 months
300 similar-quality tasks	20,000 similar-quality tasks

Human experts

Human task authors and reviewers; detailed roles are documented in each sample submission.

Raw base data

Public clinical, finance, transport and forestry datasets; exact sources and dataset notes are documented in each sample submission.

What the world looks like

CPU-only isolated workspaces with editable code, staged evidence, hidden validation and gated deployment/recovery; exact environment details are documented per task.

Time, QA and difficulty calibration

Approximately 8-10 for authoring and additional QA hours occur per task.

Sample	Claude Opus 5	Additional calibration / note
Asof Feature Leakage	0/4 strict passes	Ultra-hard long-horizon tier; reference solution passes. Agent time budget set to 7200s.
Post-PCI Clinical Case Retrieval Drift	0/4 strict passes	Ultra-hard long-horizon tier at an 100-step budget; reference solution passes.
NYC Trip Duration Serving	2/4 strict passes	Discriminates cleanly between offline reasoning and oracle-probing.
Readmission Alerting	2/4 strict passes	Strong discrimination on the subtle out-of-fold target-encoding requirement.
Covertypes District Rollout	1/4 strict passes; mean reward 0.68	Anti-cheat controls verified; metric-gaming policies are correctly blocked.

QA process

Every task passes through a systematic, automated QA pipeline, not an ad-hoc review. A layer of deterministic controls mechanically screens each task for the failure modes that undermine training value: reward-hacking and gameable or forgeable graders, answer leakage and agent-reachable oracles, grading that breaks process isolation, label-proxy features, and false difficulty (a "failure" that was only budget truncation, or a whole batch pinned at an identical floor that signals a broken grader). Each control is two-condition validated: it must fire on a deliberately planted defect and stay silent on a clean task, so the checks are proven rather than assumed.

On top of the static screen, an adversarial control battery executes cheat submissions (do-nothing, output- and ledger-forgery, and task-specific shortcuts) against the real grader in the served environment, and requires every one to score zero, while a gold reference solution and a structurally different correct solution both score full marks (grading is by outcome, not by the shape of one fix). An independent review pass grounds every finding in the actual task files (no fabricated citations), and difficulty is calibrated by running a frontier model at a fixed sampling standard inside the real isolated environment, with validity guards that separate genuine capability failures from budget/truncation artifacts. The pipeline runs automatically and reproducibly; full per-task QA evidence and rollout records accompany each sample.

Verification

Behavior-based grading against hidden or rebalanced data, plus an environment-owned terminal or published figure, required operational gates and staged-fault recovery. Network access is disabled.

Per-task deep-dive

Task	What it is about	Deliverable
Asof Feature Leakage	Point-in-time return attribution in a repeat-purchase feature pipeline.	Correct deployed pipeline and successful gated submission.
Post-PCI Clinical Case Retrieval Drift	Clinical retrieval drift across eligibility, provenance, ranking and release identity.	Correct retriever and verified release.
NYC Trip Duration Serving	Serving loses location, identifier, timestamp, timezone and DST context.	Corrected serving candidate.
Readmission Alerting	A published evaluation figure is wrong because four pipeline requirements are violated.	Corrected evaluation pipeline under both grading conditions.
Covertime District Rollout	A cover-type model fails on a held-out district despite healthy visible metrics.	Corrected pipeline clearing held-out class bars and completing release.

Example prompts

Each sample’s example prompt(s) are written up in its accompanying task submission document.

Environment and execution

Common pattern: Python CPU container, no internet, editable task workspace, task-specific CLI operations, visible tests, deterministic verifier and reference solution. See each sample document for exact dependencies, compute limits and commands.

Grading design

Component	Overview
Technical result	Hidden value, metric or terminal compared with a verifier-owned reference.
Operational path	Required action sequence replayed from a protected ledger.
Recovery	Staged fault must be diagnosed and cleared with the matching recovery family.
Isolation	Candidate code runs without access to hidden grader assets or reference outputs.

How the tasks were created

Public datasets and realistic ML failure modes were screened by measurement, then packaged with reference solutions, alternate positives, negative controls, protected verification and deterministic build checks. Detailed creation notes and supporting artifacts are provided with each sample.

Acknowledgement

I confirm that I have reviewed the tasks and believe they are realistic, technically substantive, clearly specified, and appropriately graded. They reflect genuine professional workflows and are not low-effort, artificial, or “slop” tasks.

Name: Suleiman Mahmood	Role: Co-Founder
Signature: /s/Suleiman Mahmood	Date: 8.18.2026